INGEGNERIA DEL SOFTWARE
A.A. 2025-26

PROCESSO SOFTWARE E MODELLI DI PROCESSO - 2

Docente: Michele Marchesi

marchesi@unica.it

Lucidi del corso su elearning.unica.it

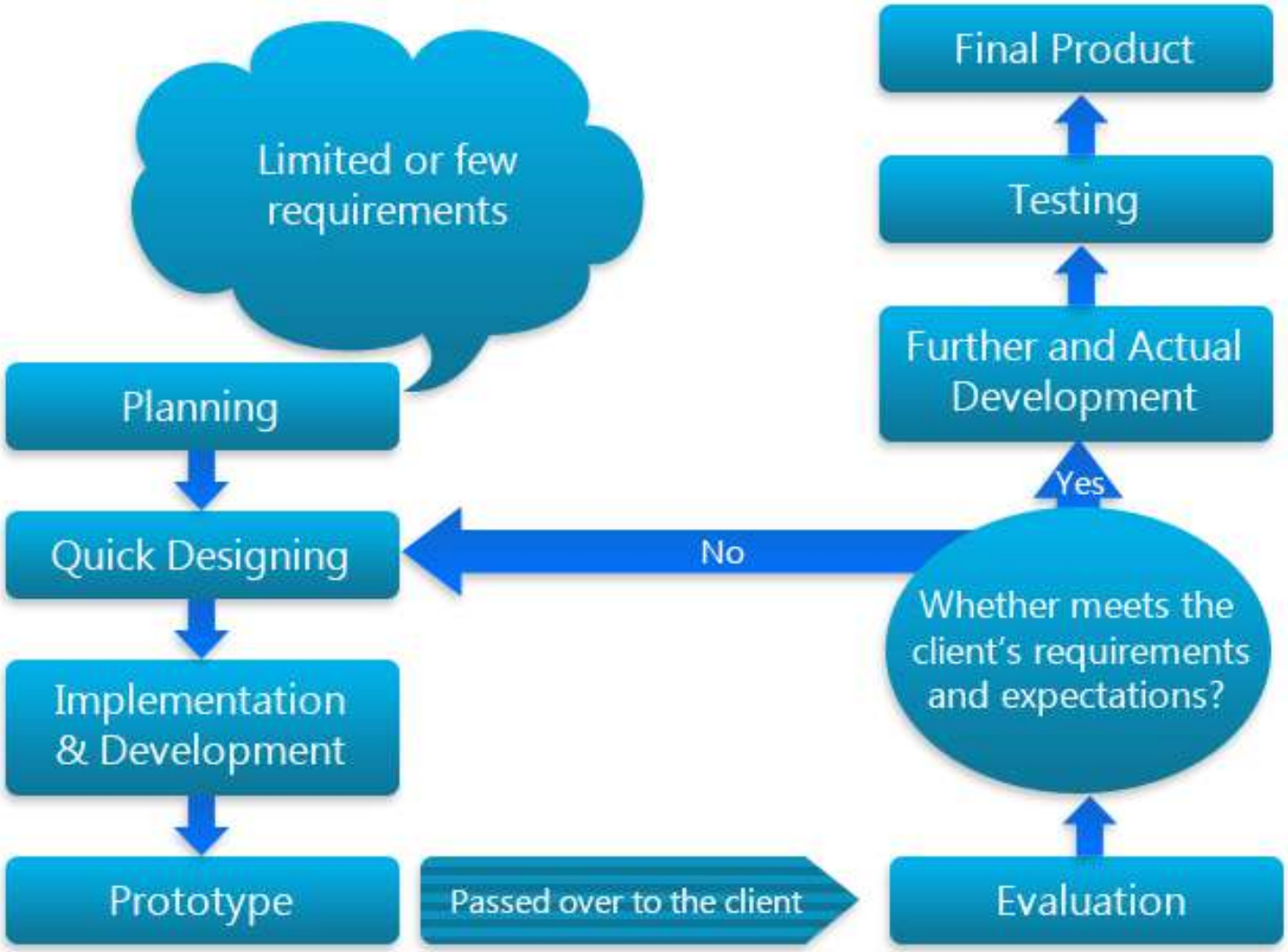
Modelli evolutivi: Prototyping

- Lo sviluppo avviene usando “prototipi”, cioè implementazioni (molto) parziali, o solo “mockup” del sistema
- Prototyping evolutivo: il prototipo evolve nel sistema finale
- Prototyping “usa e getta” (throw-away): il prototipo serve a definire bene i requisiti, poi è scartato
- L’Operational Prototyping, proposto da Ala. Davis nel 1992, integra i due approcci

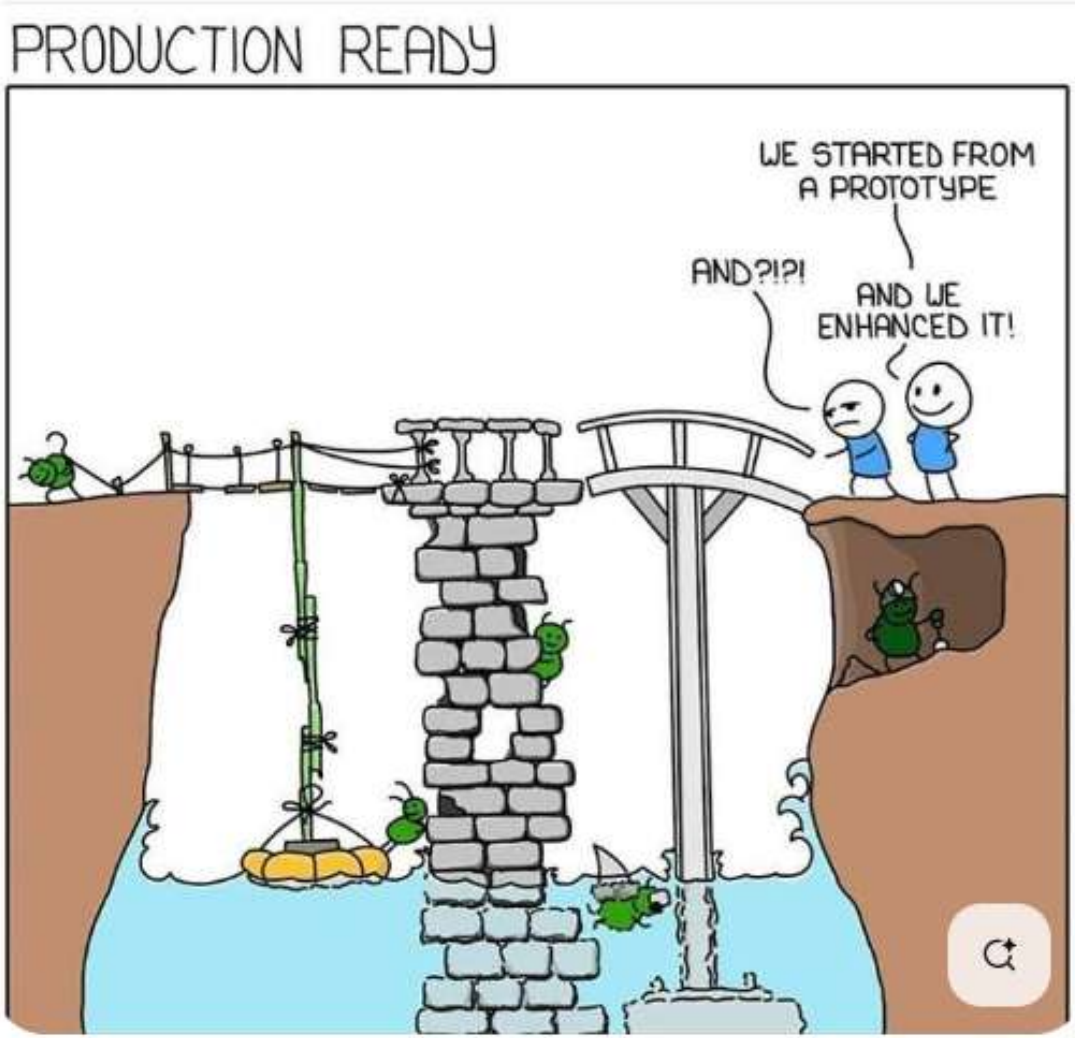
Prototyping

- Usato se il cliente ha un'idea, ma non ne sa definire i requisiti
- Il cliente:
  - comunica l'idea al team
  - il team progetta e costruisce in fretta un prototipo (non un sistema funzionante!):
    - mostra solo la GUI, senza alcuna funzionalità
    - può implementare qualche funzione
  - questo è mostrato al cliente, che dà un feedback
  - il ciclo è ripetuto sino a stabilire i requisiti, talora sino allo sviluppo del sistema

Prototyping



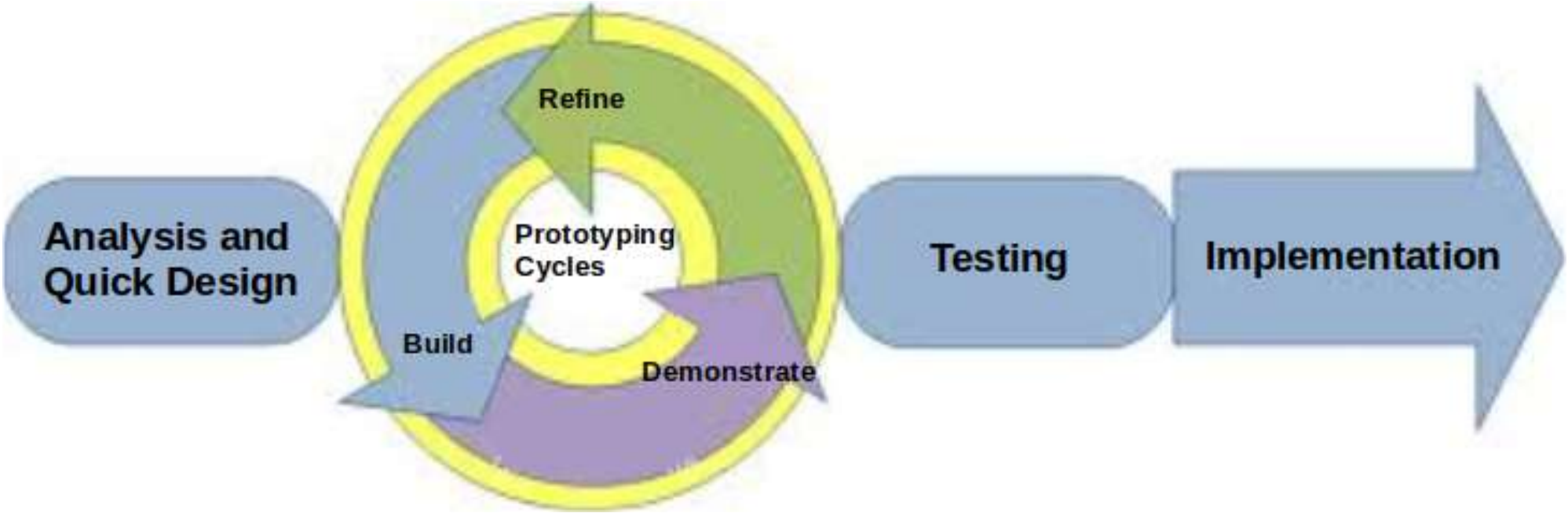
Prototyping… evolutivo



RAD - Rapid Application Development

- Formalizzato da James Martin (IBM) nel 1991
- Simile al Prototyping
- Fasi:
  - Raccolta dei requisiti, analisi e progetto iniziali
  - Cicli di prototipizzazione:
    - progetto fatto insieme agli utenti (User Design)
    - costruzione/miglioramento del prototipo
    - feedback dell'utente
  - Cutover: ingegnerizzazione finale, testing, rilascio, addestramento degli utenti finali
- Uno dei precursori delle Metodologie Agili

Rapid Application Development



RAD - Vantaggi

- Tempi di sviluppo ridotti e maggiore produttività rispetto al Waterfall
- Il coinvolgimento dei clienti per tutto il ciclo riduce il rischio di non soddisfare i clienti
- Il centro si sposta dalla documentazione al codice (*Quello che vedi è quello che ottieni*)
- Utilizza tecniche di modellazione esplicita per acquisire informazioni sul modello di business, sui dati e sui processi

RAD - Svantaggi

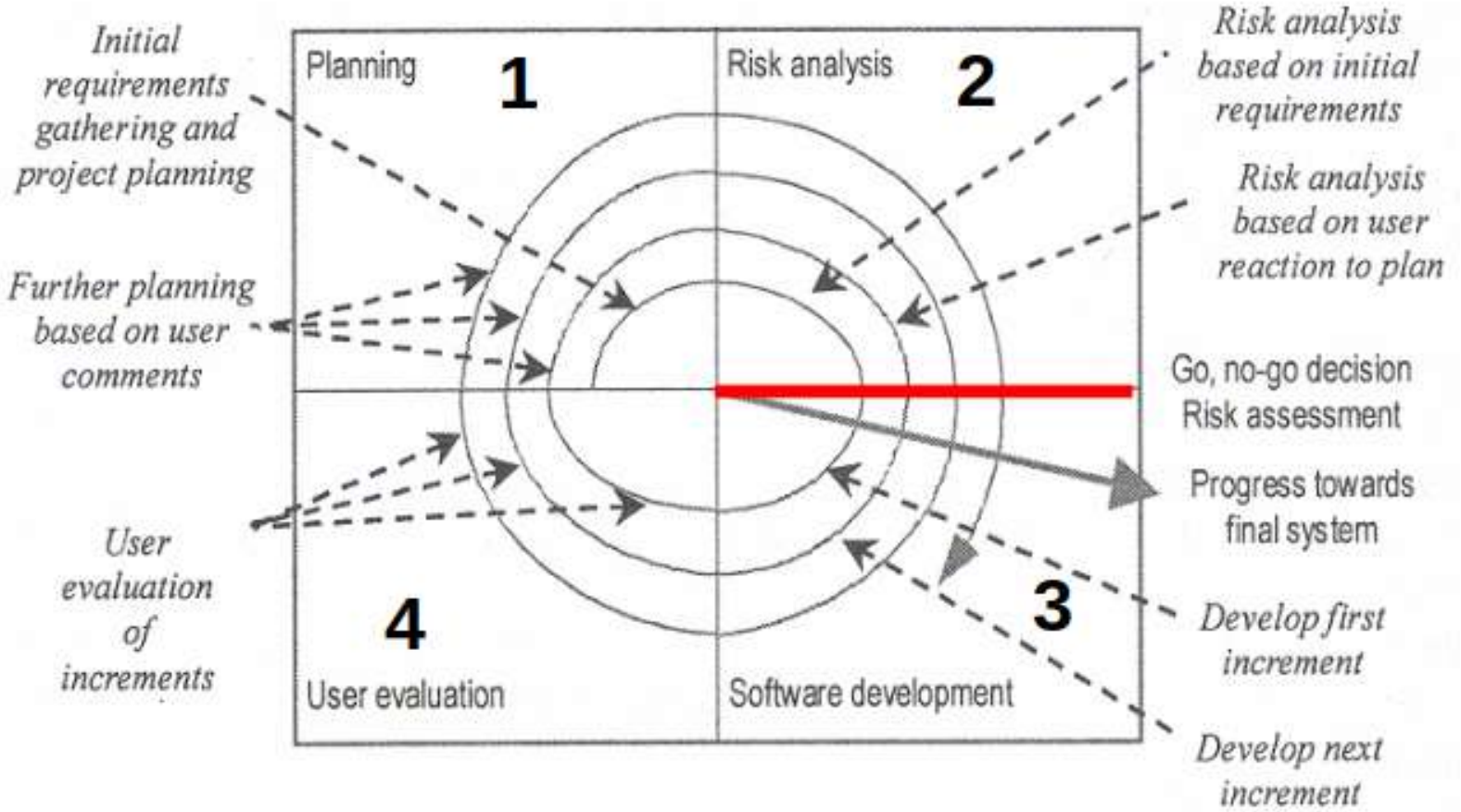
- Difficile stimare i costi complessivi e i tempi di rilascio
- Le iterazioni brevi possono non permettere di aggiungere funzioni complesse
- Scalabilità ridotta perché il sistema inizia come prototipo, ed evolve nel prodotto finito
- Forte dipendenza dalla qualità e coesione del team

RAD – Va applicato quando:

- I requisiti sono ragionevolmente ben noti
- L'utente si può coinvolgere per tutto il ciclo di vita
- Le funzionalità si possono rilasciare in modo incrementale
- Non sono richieste elevate prestazioni
- I rischi tecnici sono bassi
- Il sistema può essere modularizzato

Il modello a spirale (Boehm 1988)

- Approccio evolutivo con interazioni continue fra cliente e developer
- Modello "**risk driven**": ***tutte le scelte sono basate sui risultati dell'analisi dei rischi***



I quattro settori della spirale

1. **Pianificazione**
   - Identifica obiettivi specifici per una fase (un giro)
   - Pianificazione della fase
2. **Valutare e ridurre i rischi**
   - I principali rischi sono identificati, analizzati, e viene raccolta informazione su come minimizzarli

- ***** DECISIONE SE PROSEGUIRE O NO *****

3. **Sviluppo e validazione**
   - Viene scelto un modello di sviluppo e validazione dei risultati
   - In genere, è un piccolo “Waterfall”
4. **Feedback**
   - Review del progetto e requisiti per il nuovo ‘giro’

Il rischio in Ingegneria del Software

- Un **rischio** è una circostanza **potenzialmente avversa** in grado di **pregiudicare** lo sviluppo e/o la qualità del software
- Ogni scelta/decisione → un rischio
- **Il rischio maggiore**: ***probabilità di non riuscire a implementare tutte le funzionalità richieste e di rispettare tutti i requisiti***
- **Altri rischi**: probabilità di ***non rispettare tempi*** di consegna o il ***budget*** di progetto
- Il rischio si tiene sotto controllo con:
  - attività esplorative
  - verifiche periodiche

Metodi Formali per la specifica, lo sviluppo e la verifica del software

- Insieme di tecniche logico-matematiche che permettono **l'esplorazione esaustiva di tutti i possibili comportamenti del sistema**
- Obiettivo dei metodi formali - Fornire:
  - la capacità di esprimere specifiche in maniera precisa;
  - la capacità di definire in modo chiaro quando un’implementazione soddisfa le specifiche ("correttezza");
  - eventualmente, lo sviluppo del software per compilazione diretta delle specifiche
- Sono molto costosi da applicare e non sono adatti per sistemi grandi e complessi

Metodi Formali: applicazioni

- I Metodi Formali sono applicati in caso di:
  - sistemi *life-critical* o *mission-critical* di dimensioni medio-piccole
  - nuclei operativi di tali sistemi
- Ad es.
  - Sistemi di autenticazione
  - Pacemaker
  - Sistemi di controllo critici (nucleare, avionica, ferroviario, biomedicale, ecc.)

Il “Rational Unified Process” (RUP)

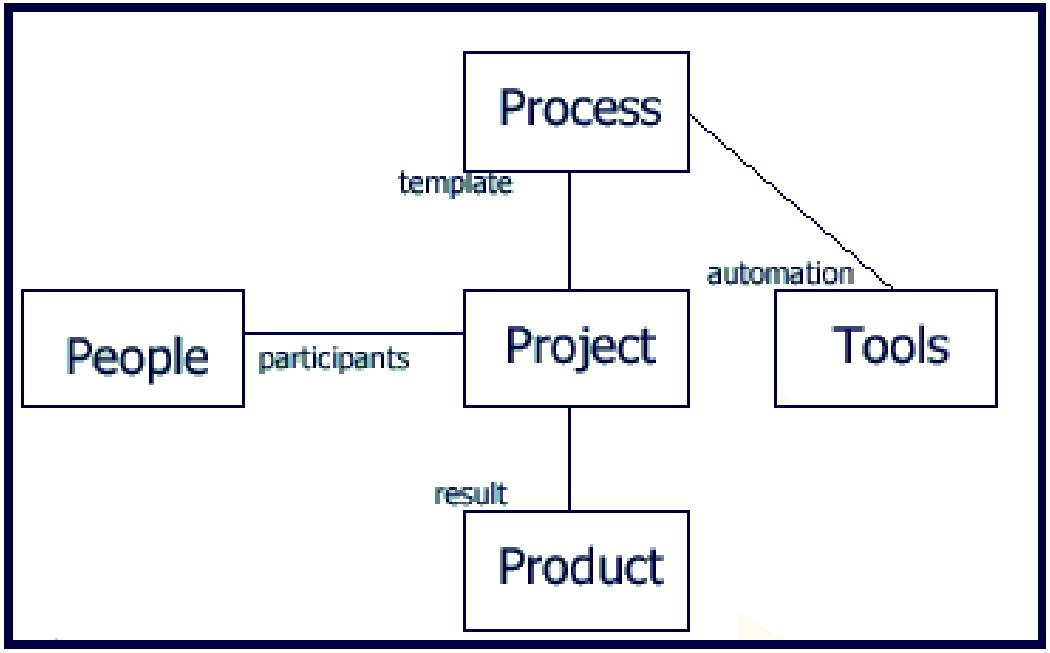
- Tentativo di sviluppare un processo che vada bene per tutti i progetti software che usano tecnologie ad oggetti
- Si basa sulla notazione UML (Unified Modeling Language)
- Il RUP è:
  - **guidato dai casi d’uso**
  - **centrato sull’architettura**
  - **incrementale e iterativo**
- E’ supportato da strumenti CASE, in particolare della Rational (ora IBM Engineering Rhapsody)
- E’ in effetti un **meta-processo**, che offre un ventaglio di attività, ruoli, artefatti, *milestone*, … configurabili in uno specifico processo

I Tres Amigos”

- Autori del linguaggio UML (sarà ampiamente trattato nel corso) e del RUP:

I quattro P (+T)

- 4 aspetti fondamentali per il RUP:
  - Persone (People)
  - Progetto
  - Prodotto
  - Processo
  - (Tools)



Perché i 4 P

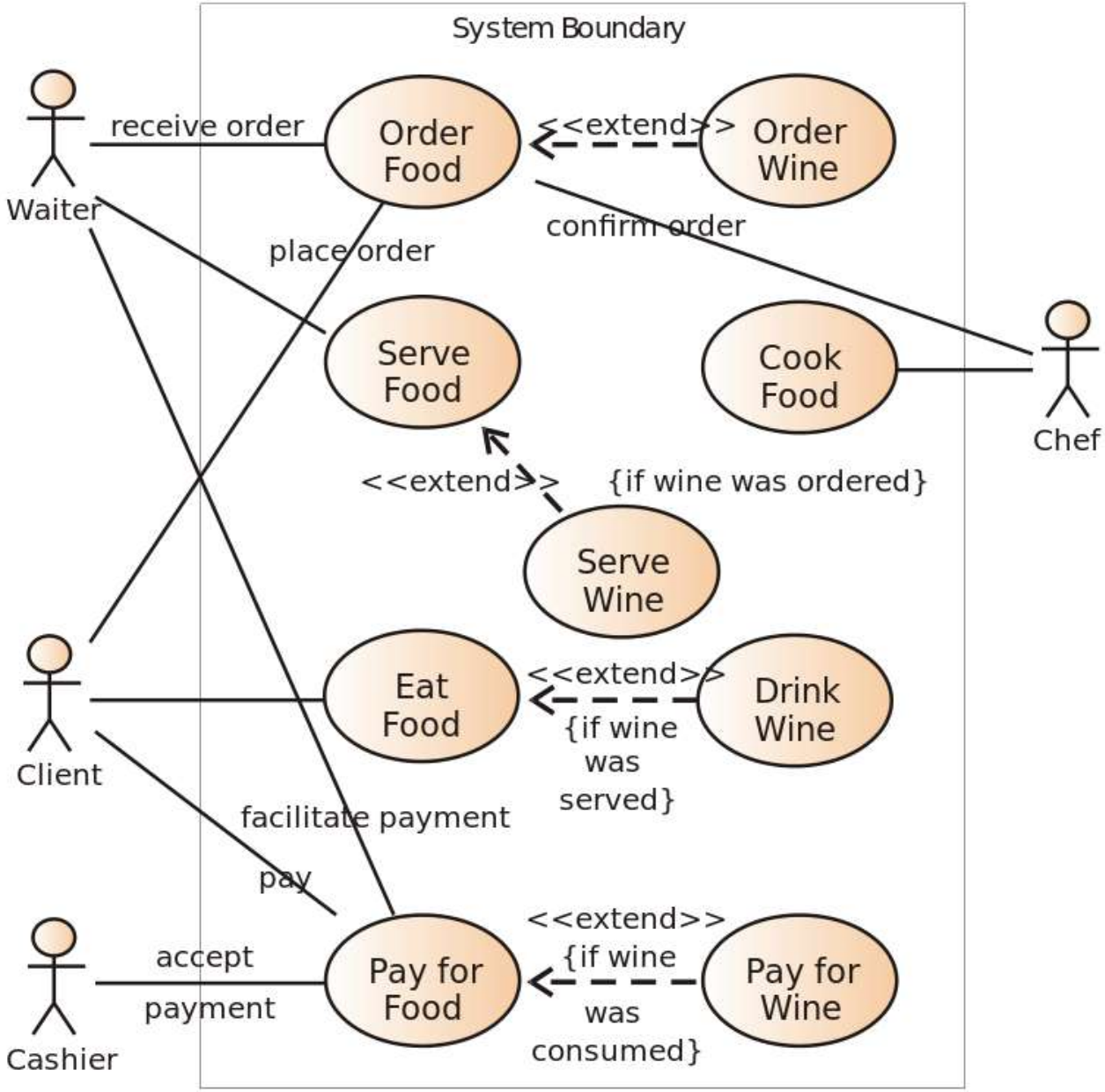
- Le Persone sono alla base di tutto
- Il Progetto produce il Prodotto
- Il Prodotto non è solamente il codice, ma tutti i documenti e il materiale di supporto
- Il Processo guida i Progetti
- Gli strumenti (Tools) sono fondamentali per aiutare il Processo

Guidato dai casi d'uso (CdU)

- Un *caso d'uso* o *Use Case* specifica una sequenza d'azioni, comprese le varianti, che il sistema esegue in risposta ad un input e che produce un risultato di valore tangibile per un dato *attore* (*…* li vedremo bene in seguito!)
- I CdU sono un mezzo sistematico e intuitivo per raccogliere i requisiti funzionali
- I requisiti del sistema sono quindi una lista di CdU
- Lo sviluppo del sistema avviene implementando i vari CdU

Un diagramma UML dei casi d'uso

(da Wikipedia)



Centrato sull'architettura

- L'**Architettura** di un sistema software secondo il RUP è:
  - L'organizzazione del sistema
  - L'insieme degli elementi strutturali (**dati**) e comportamentali (**funzioni**), e delle **interfacce** che costituiscono il sistema
  - La composizione di elementi strutturali e comportamentali (moduli) in sistemi progressivamente più grandi
- L'architettura si sviluppa iterativamente soprattutto durante la fase di *elaborazione*
- Si inizia da un'architettura iniziale, derivata dai CdU significativi per l'architettura e da altri input

Le 4 fasi del RUP

- **Inception Phase (fase iniziale)**
- **Elaboration Phase (fase di elaborazione)**
- **Construction Phase (fase di costruzione)**
- **Transition Phase (fase di transizione)**

Si noti che tutti tipi di "workflow" (attività) sono effettuati in tutte le 4 fasi, sia pur con diversi gradi di intensità!

Fase iniziale (Inception)

- Si può considerare come un'elaborazione e precisazione del concetto generale di **analisi di fattibilità**
- Lo scopo principale è quello di delineare in modo accurato il ***business case***, ovvero comprendere il tipo di mercato o di esigenza al quale il progetto afferisce e identificare gli elementi chiave per condurlo al successo (commerciale o no)
- Fra gli strumenti utilizzati ci sono:
  - un elenco dei casi d'uso
  - la pianificazione iniziale del progetto
  - la valutazione dei rischi
  - una definizione grossolana dei requisiti
- Se il progetto non supera questa milestone, detta "**Lifecycle Objective Milestone**", esso dovrà essere abbandonato o ridefinito.

Fase di elaborazione (Elaboration)

- Definisce la struttura complessiva del sistema
- Comprende l'**analisi di dominio** e una prima fase di **progettazione dell'architettura**
- Si conclude col superamento di una milestone detta "**Lifecycle Architecture Milestone**", soddisfacendo i seguenti criteri:
  - sviluppato un modello dei casi d'uso completo all'80%
  - fornita la descrizione dell'architettura del sistema
  - sviluppata un'"architettura eseguibile" che dimostri il completamento degli use case significativi
  - eseguita una revisione del business case e dei rischi
  - completata una pianificazione del progetto complessivo
- Se il progetto non passa questa milestone, potrebbe ancora essere abbandonato, oppure dovrà essere rivisto
- Al termine di questa fase si transita in una situazione di rischio più elevato, in cui le modifiche all'impostazione del progetto saranno più difficili e onerose

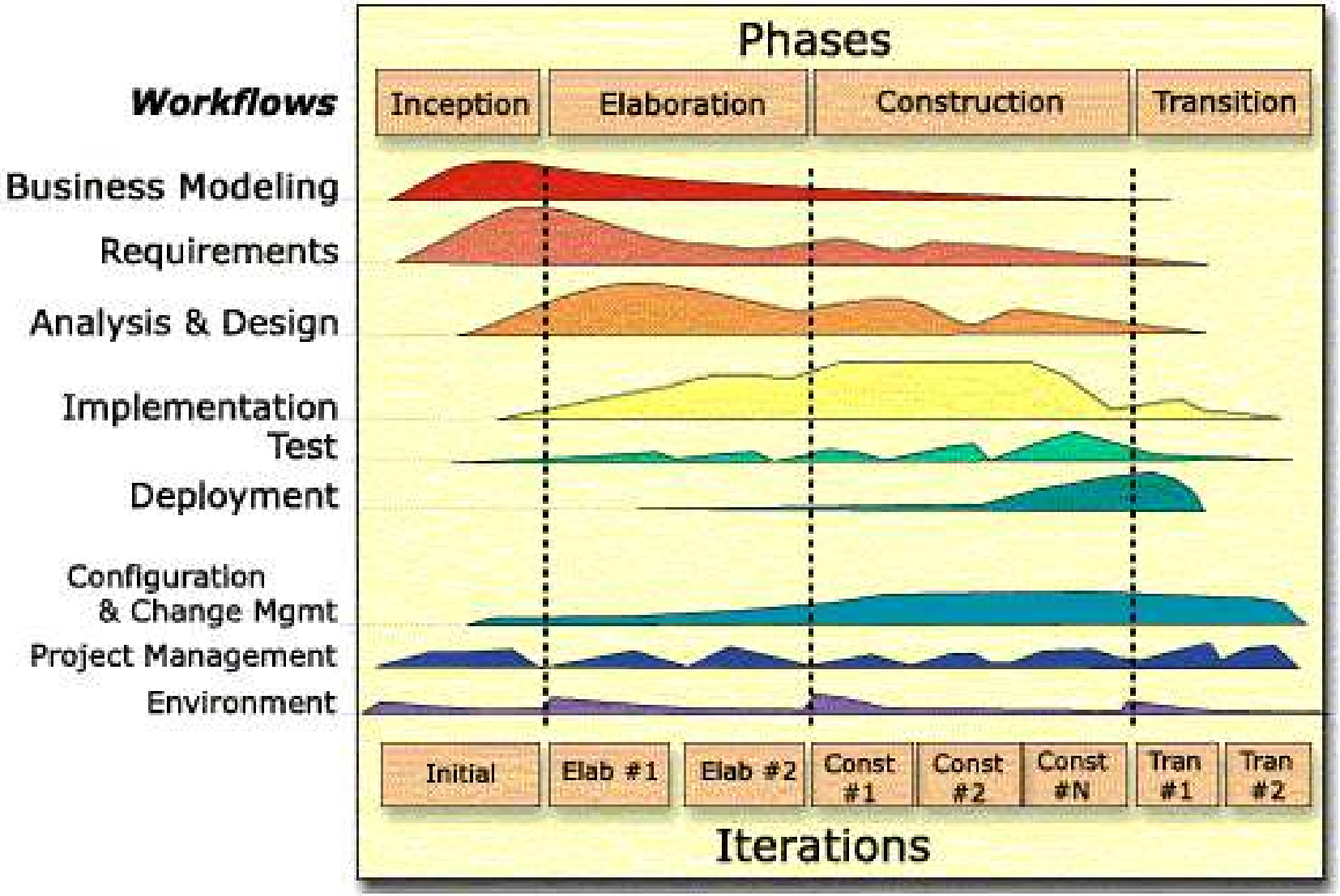
Fase di costruzione (Construction)

- L'obiettivo principale è costruire il sistema
- Ci si concentra sullo sviluppo dei componenti (moduli) e delle altre funzionalità del sistema
- In questa fase viene portato a termine il grosso dello sviluppo
- Nei progetti più grandi, ci possono essere molte iterazioni di sviluppo, cercando di dividere i Casi d'Uso in sottoinsiemi gestibili che producano prototipi dimostrabili al cliente
- Viene prodotta la prima release del sistema
- La milestone di questa fase si chiama "**Initial Operational Capability**" e rappresenta la prima disponibilità delle funzionalità del sistema in termini di implementazione

Fase di transizione (Transition)

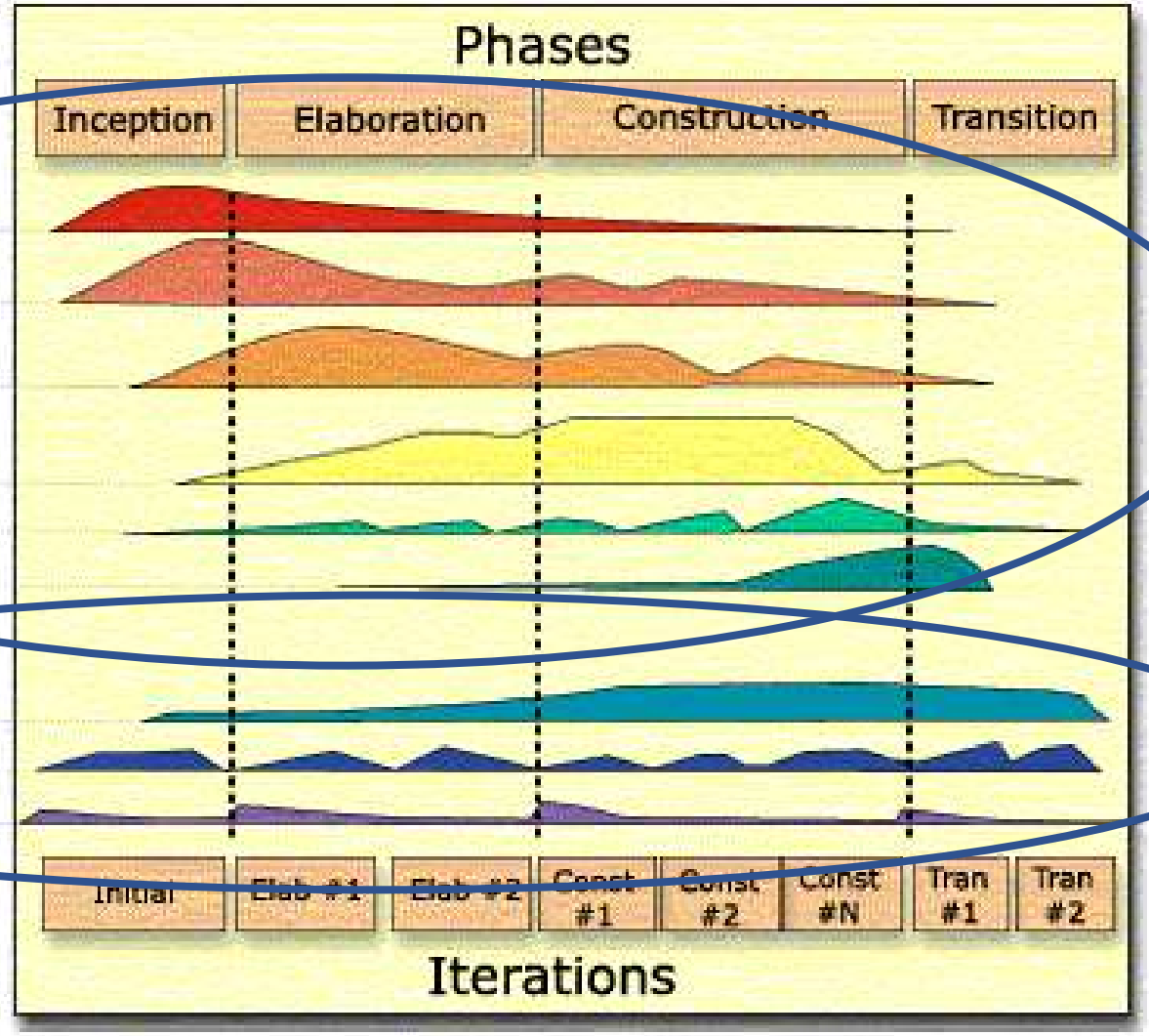
- Nella fase di transizione, il sistema passa dall'ambiente dello sviluppo a quello del cliente finale (*Operations*)
- Vengono condotte le attività di training degli utenti e il beta testing del sistema a scopo di verifica e validazione
- Si deve in particolare verificare che il prodotto sia conforme alle aspettative descritte nella fase di Inception
- Se questo non è vero si procede a ripetere l'intero ciclo
  - altrimenti, si raggiunge la milestone detta "**Product Release**" (Rilascio) e lo sviluppo termina

Schema del RUP



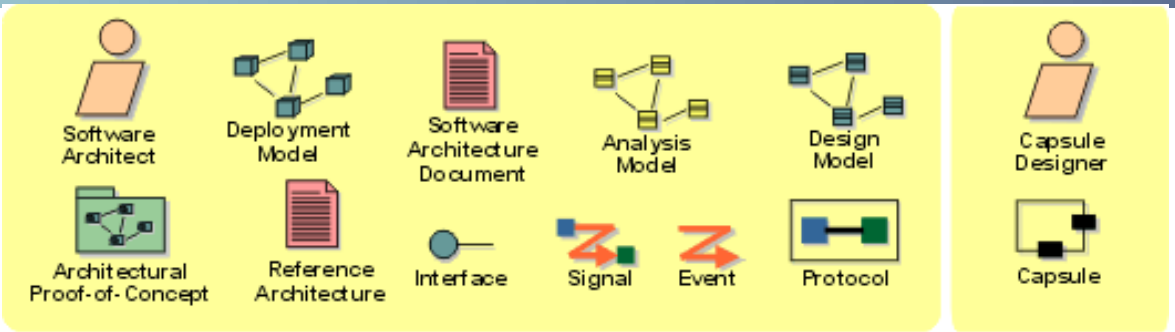
Schema del RUP

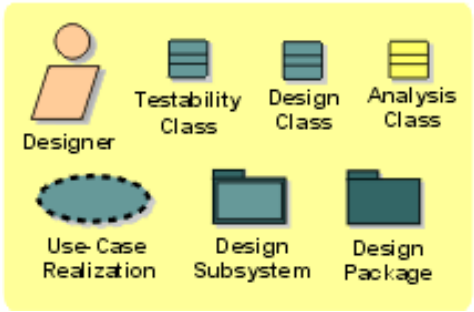
Workflow di processo (tecnici)

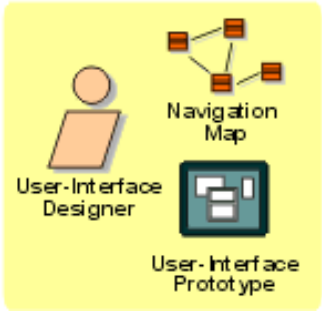


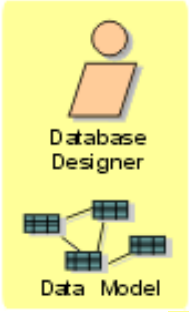
Workflow di supporto (gestionali)

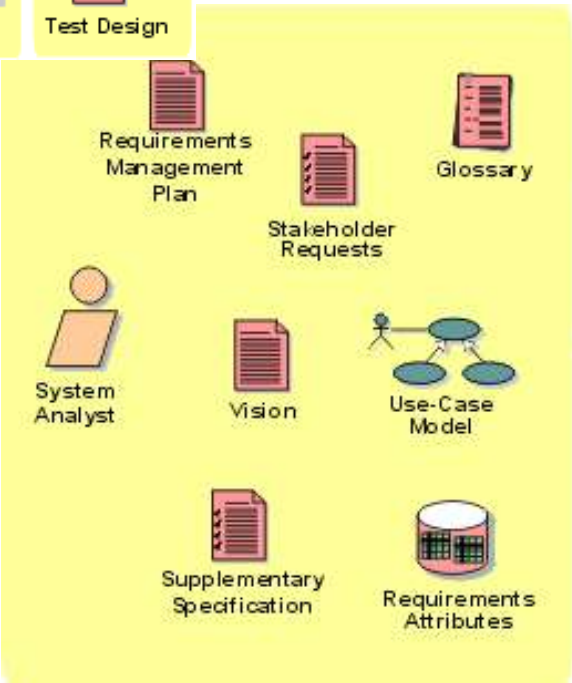
Ruoli, manufatti, notazione del RUP

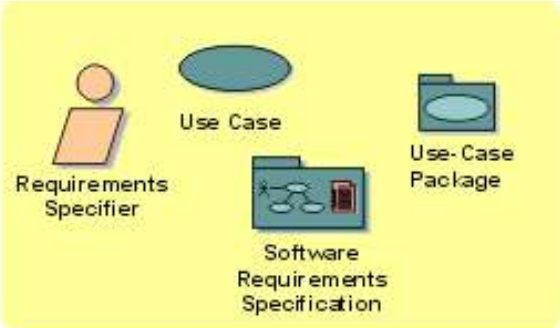


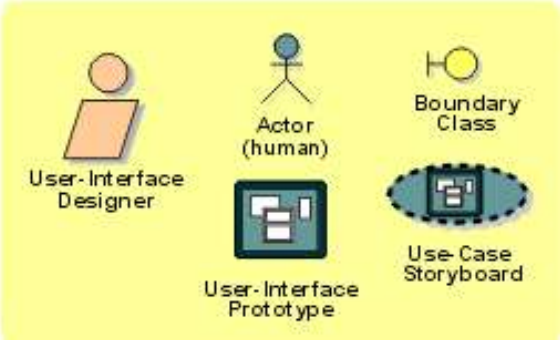




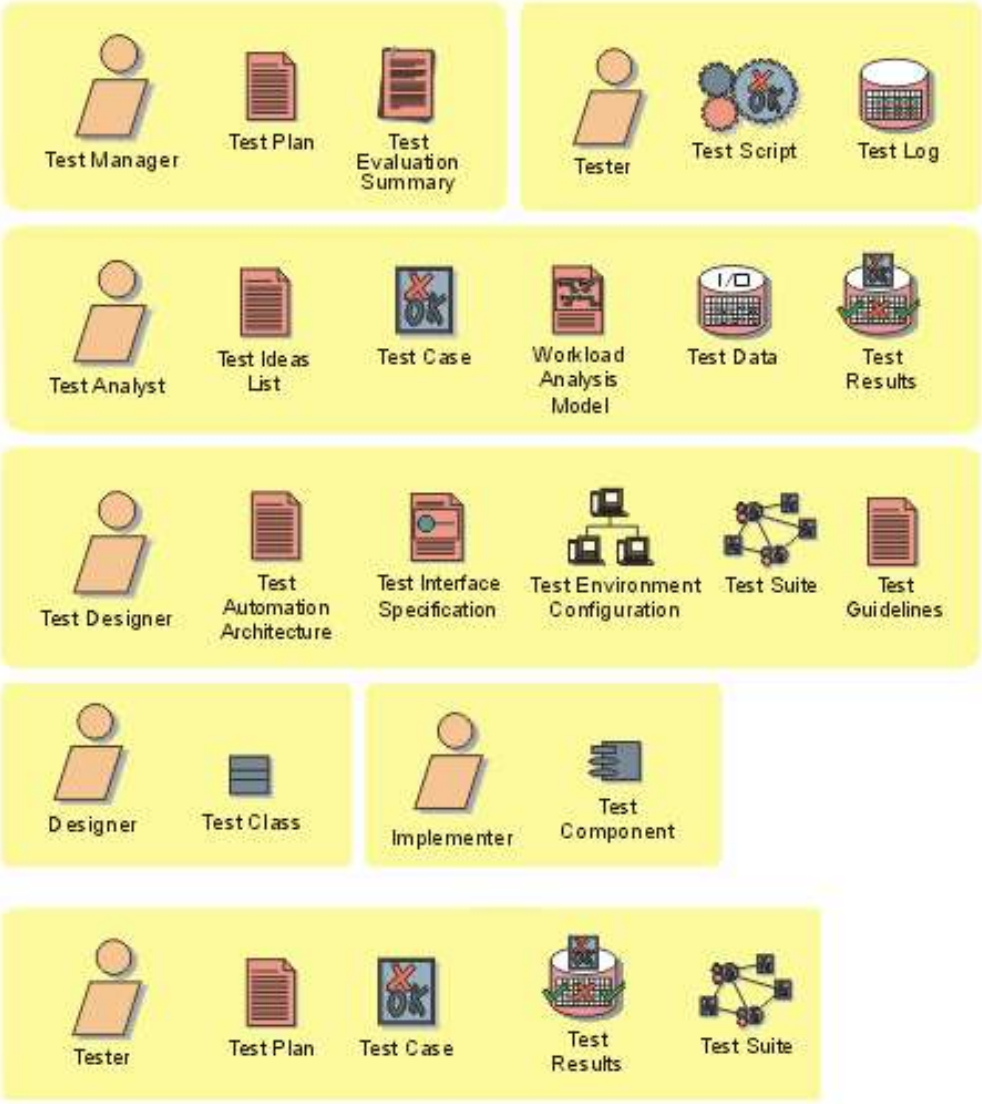


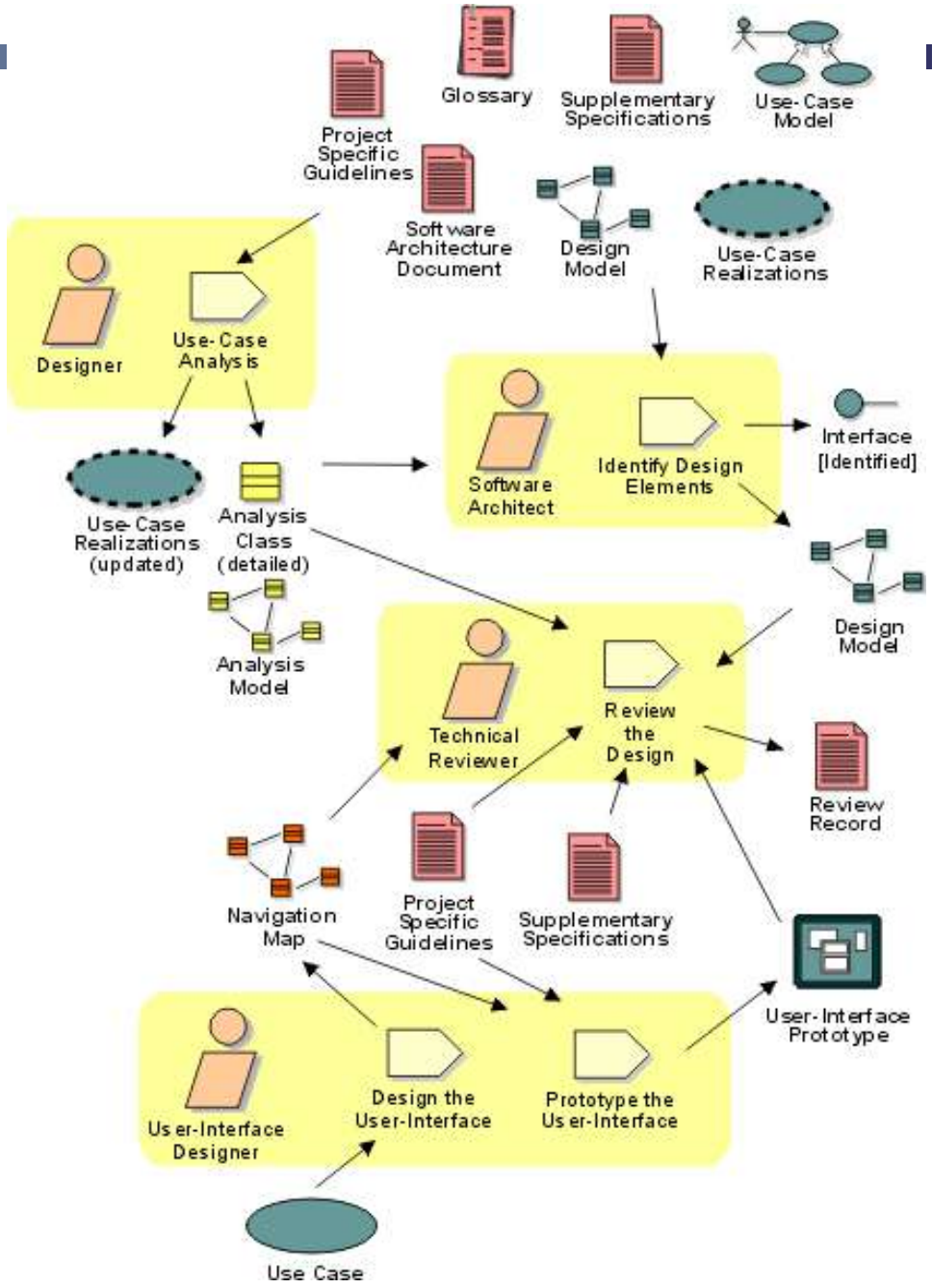




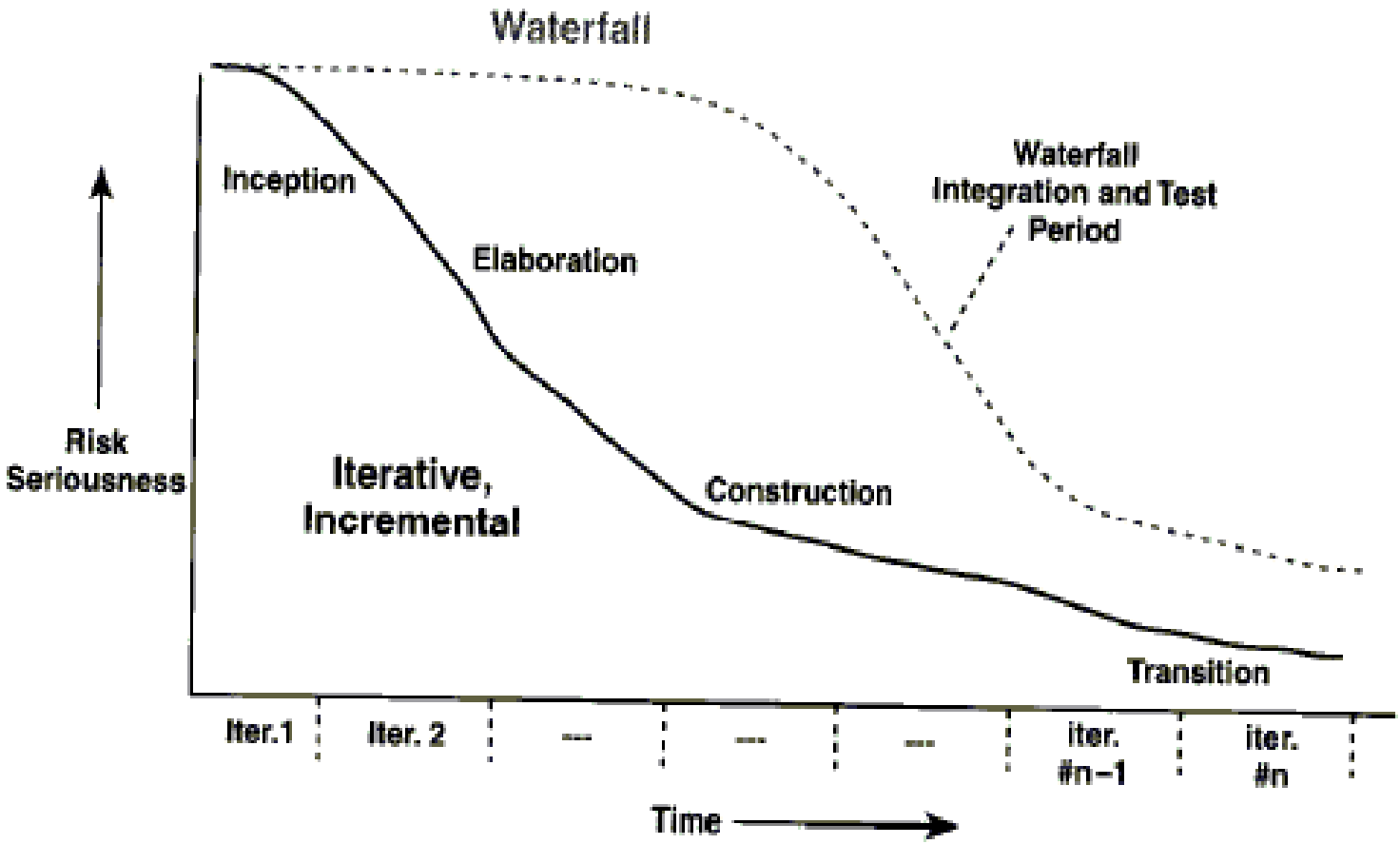


Ruoli, manufatti, notazione del RUP

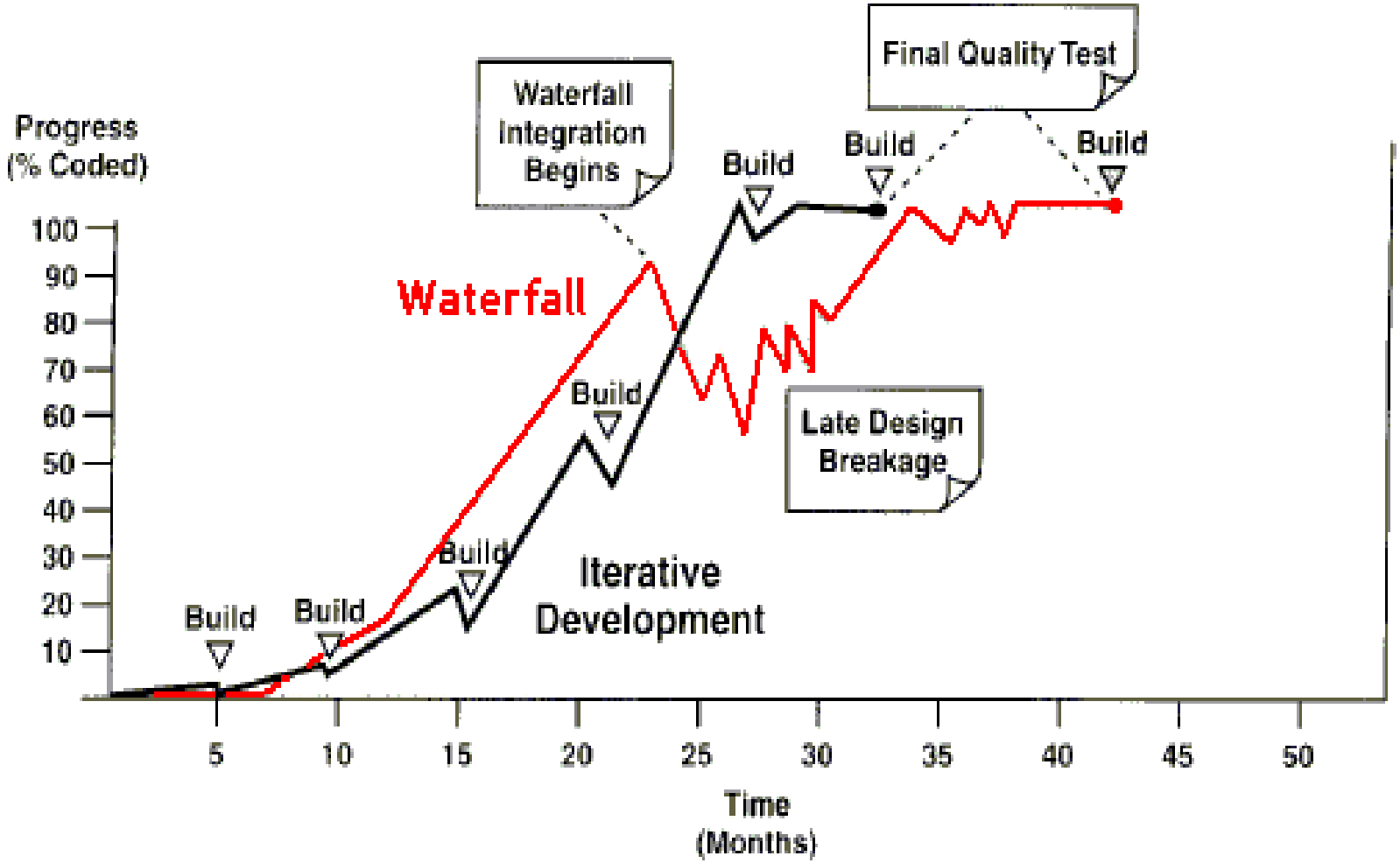




RUP e Waterfall: livelli di rischio



RUP e Waterfall (% di completamento)



Le 6 “Best Practices” del RUP

- Sviluppo iterativo (e incrementale)
- Gestione dei requisiti (tracciabilità)
- Uso di componenti (riusabili)
- Modellazione visuale (con UML)
- Verifica della qualità (testing)
- Controllo dei cambiamenti (integrazione continua)

RUP: pregi e difetti

Pregi

- Introduce le iterazioni in un processo “mainstream”
- Si basa sui Casi d'Uso
- E' configurabile secondo le esigenze di chi sviluppa
- Molta attenzione alla tracciabilità dei requisiti e al livello di “business”
- Piuttosto diffuso

Difetti

- E' molto “pesante”: troppi ruoli, attività, manufatti
- Difficile da imparare bene
- Troppo legato a strumenti CASE costosi
- Gestione di progetti reali efficace ma “pesante”
- Non cura molto l'interazione col cliente

Concetti dell'approccio classico (BDUF)

- Modificare un sistema software in fase di avanzato sviluppo è troppo costoso: meglio fare tutti gli sforzi possibili per *farlo giusto sin dal principio!*
- → **BDUF = Big Design Up-Front**
- Occorre portare anche nel software le tecniche dell'ingegneria classica
- Sviluppo per componenti
- Progettare per il riuso

Un'analogia: la costruzione di un grattacielo

- **Prima di costruire**, occorre fare **dettagliati progetti** architettonici e strutturali
- Architetti e ingegneri progettisti **specializzati** eseguono il progetto
- La costruzione è affidata a tecnici e operai di cantiere, ***secondo i dettami del progetto***
- Il costo di progetto non supera il **5-10%** del costo totale
- Il grattacielo è ***fatto giusto sin dal principio***!

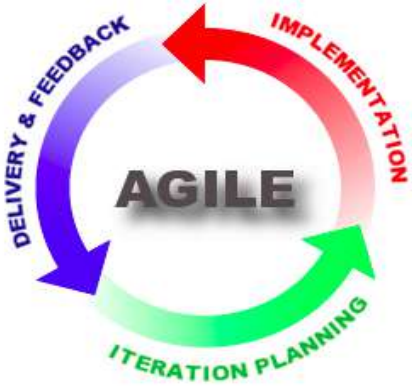
Ma il software non è un grattacielo!

- Il software non è materiale, quindi è possibile modificarlo anche in corso d'opera
- I requisiti funzionali e tecnologici cambiano in continuazione, *alla velocità di Internet!*
- La scrittura del software a partire dai diagrammi di progetto è un'**attività creativa** e non una semplice esecuzione
- Il costo delle fasi di analisi e progettazione può superare il **50%** del costo totale di sviluppo

Problemi dell'approccio classico

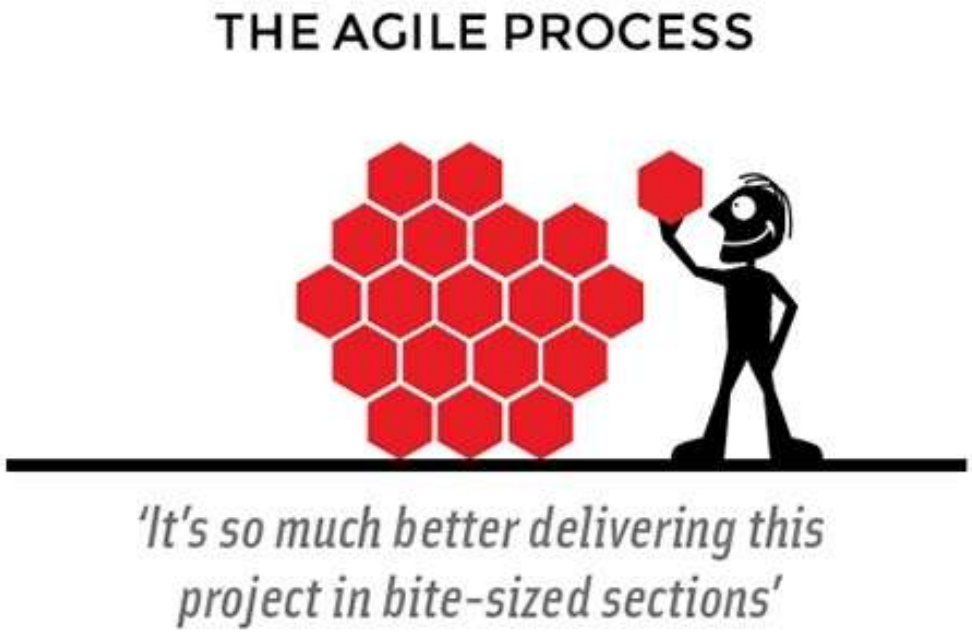
- L'approccio classico vale per sistemi con requisiti molto stabili
- Il tempo tra la raccolta dei requisiti e l'inizio della codifica può essere anche di 1-2 anni
- Nell'era dell'e-business:
  - Il "**time-to-market**" è dell'ordine dei mesi o delle settimane
  - I requisiti cambiano in continuazione
  - L'incertezza è molto alta

Una possibile risposta: le Metodologie Agili (MA)



- Adattative e non predittive
- Orientate alle persone e non al processo
- Mantenute semplici per poter reagire in fretta
- Incrementali e iterative, con iterazioni molto corte
- Ruoli intercambiabili degli sviluppatori
- Basate su testing e codifica e non su analisi e progetto

Waterfall vs. Agile



Metodologie Agili per lo sviluppo del software

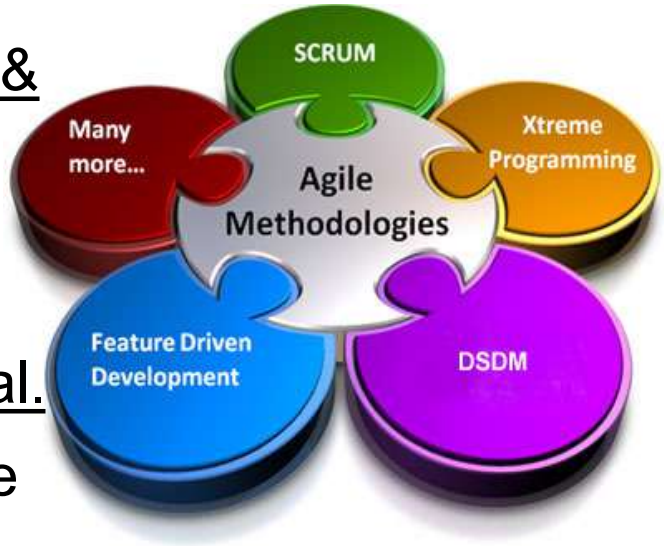
- **Iterative**
  - procedono per iterazioni brevi e fissate
- **Incrementali**
  - sviluppano il sistema implementando una lista di funzionalità ad ogni iterazione
- **Cooperative**
  - sviluppatori e clienti comunicano in continuazione
- **Chiare**
  - facili da imparare e modificare, ben documentate
- **Adattative**
  - consentono cambiamenti in corso d'opera

Principali pratiche Agili

- Sviluppo incrementale guidato da “features”
- Enfasi su requisiti spiegati verbalmente
- Analisi e progetto fatte minimizzando l'uso di diagrammi formali
- Minima documentazione oltre al codice
- Testing automatico e continuo
- Iterazioni brevi e integrazione continua
- Refactoring, semplicità
- ...

Metodologie Agili

- **Scrum** (introdotta nel 1990), di Ken Schwaber & Jeff Sutherland
- **Extreme Programming (XP)** di Kent Beck (la più diffusa inizialmente)
- **Lean Software Development** di G. Howell et al.
- **Feature Driven Development (FDD)** di Jeff De Luca & Peter Coad
- **Dynamic Systems Development Method (DSDM)**
- **Crystal Methods** di Alistair Cockburn
- **SAFe** (Scaled Agile Framework): pensato per organizzazioni più grandi che cercano di usare le pratiche agili con più team.
- Vedremo in seguito le prime 3



Esempio: sistema BDUF

- Progetto SACWIS dello stato della Florida
- Gestione assistenza statale all'infanzia
- Iniziato nel 1990
- Sistema molto grande e completo
- Budget stimato: **32 M$**
- Data di consegna stimata: 1998
- Staff di **109 persone** sul progetto

SACWIS Florida: risultati

- **170 M$** spesi al 2002 (oltre 5 volte il budget)
- Nuova stima dei costi: **230 M$** (7 volte il budget)
- Nuova data di consegna: 2005
- ***Progetto cancellato!***
- ***... un bagno di sangue!***
- Nel 2007-2009 la Florida ha adottato il SACWIS del Wisconsin, basato su Java, Windows, Linux e Oracle

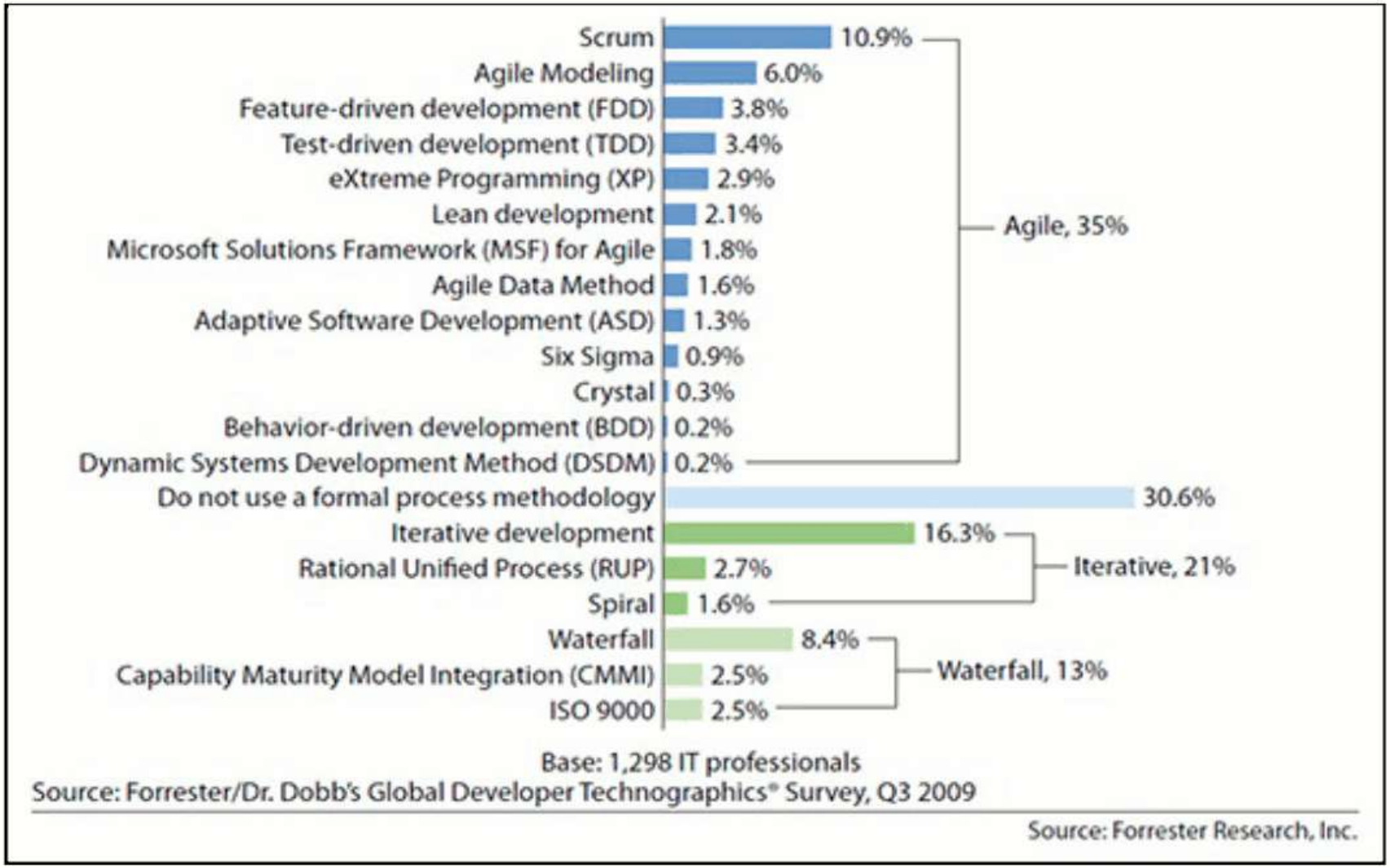
Esempio: sistema Agile

- Progetto SACWIS dello stato del Minnesota
- Iniziato nel 1999
- Approccio: "***fa la cosa più semplice che possa funzionare***"
- Sistema che soddisfa i **requisiti minimi** SACWIS
- Budget stimato: 1.1 M$
- Data di consegna stimata:
  - Settembre 1999 per fase 1
  - Agosto 2000 per fase 2
- Staff di 8 persone

SACWIS Minnesota: risultati

- Completato all'inizio del 2000
- Spesi **1.1 M$**
- **Successo completo!**
- Motivazioni del successo:
  - Infrastrutture standard
  - Requisiti mantenuti minimi

Adozione delle MA (2009)



Le MA nel 2024

- **Tasso d'adozione**: il 97% delle organizzazioni dice di usare, almeno in parte, i metodi agili di sviluppo
- **Percentuali di successo**: i progetti gestiti con MA hanno una percentuale di successo del 75%; la percentuale di successo dei metodi tradizionali si aggira intorno al 56%.
- **Valore di mercato**: il mercato globale degli strumenti di sviluppo Agile raggiungerà i 9,2 miliardi di dollari entro la fine del 2024, rispetto ai 5,7 miliardi di dollari del 2020.
- Metodi più popolari:
  - Scrum continua a dominare come il metodo Agile più diffuso, utilizzato da circa il 70% dei professionisti Agile.
  - Segue Kanban, con una percentuale di utilizzo di circa il 50%.

da: https://staragile.com/blog/state-of-agile

Processi di sviluppo sw nel 2025

Metodologia/ approccio	Livello di Diffusione (dati 2025)	Note e Trend
Scrum	Metodologia più usata a livello di team	Rimane il framework fondamentale, spesso combinato con Kanban ("Scrumban")
Kanban	65% delle organizzazioni di sviluppo software	In forte crescita (dal 39%). Migliora collaborazione, trasparenza ed efficienza
Lean	40% delle organizzazioni	Utilizzato per iniziative di trasformazione, spesso integrato con AI e automazione
Modelli Ibridi	74% delle organizzazioni	Trend in forte crescita. Il 73% prevede di aumentare l'uso di ibridi nei prossimi 5 anni
SAFe (Scaled Agile Frame-work)	44% (in ripresa)	Spesso come componente all'interno di modelli ibridi
DevOps / DevSecOps	Oltre l'80%	Pratiche ormai mainstream e considerate essenziali
Waterfall (puro)	Non quantificata, ma in nicchie specifiche	Settori con vincoli normativi rigidi (aerospaziale, medicale)
RUP, XP (puri)	Non rilevati come metodologie primarie	Pratiche assorbite in framework più ampi o modelli ibridi